S9 — Consistencia distribuida en procesos de negocio

Idempotencia y compensación en una **Saga coreografiada** con Kafka: **veterinaria-pago-ms** deja de romperse ante un `cita.creada` repetido, la pasarela simulada puede rechazar un pago (`pago.fallido`) y **citas-ms** compensa la cita (`PENDIENTE_PAGO → CANCELADA`) devolviendo el insumo clínico reservado. Como actividad autónoma, una segunda saga propia —el **alta de cliente** entre **veterinaria-auth-ms** y **veterinaria-cliente-ms**— con su propio evento de compensación y liberación del email reservado.

Datos del estudiante
1. Introducción y motivación
2. Arquitectura: dos sagas coreografiadas
3. Bloque 1 — Bug de duplicados e idempotencia
4. Bloque 2 — Fallo de pago y compensación
5. Bloque 3 — Compensación idempotente
6. Parte D — Estados y transiciones de Cita
7. Parte E (bono) — Reserva y devolución de stock
8. Bloque 4 — Proceso propio: alta de cliente
9. Error o hallazgo técnico
10. Preguntas de defensa
11. Reflexión técnica
12. Criterios, limitaciones y bibliografía
Anexo: Feedback de la sesión

ESTUDIANTE
Joaquín Hassan Sanchez Saravia

SESIÓN
S9 · Consistencia distribuida en procesos de negocio

FECHA
15 de octubre de 2026

Datos del estudiante

Nombre	Joaquín Hassan Sanchez Saravia
Equipo	##
Sesión	S09 - Consistencia distribuida en procesos de negocio
Rol o aporte realizado	Implementación individual completa: idempotencia de dos capas en `veterinaria-pago-ms`, estado `FALLIDO` y evento `pago.fallido`, compensación en `citas-ms`, reserva/devolución de insumos en `veterinaria-mascota-ms` (Parte E) y la saga propia de alta de cliente (`veterinaria-auth-ms` ↔ `veterinaria-cliente-ms`), con pruebas y evidencia.
Link de GitHub	https://github.com/JoaHasss/veterinaria-mascota-ms

> **Sobre la evidencia.** Las salidas de consola y de log de este informe son copia literal de la ejecución real del 15/10/2026 entre las 19:25 y las 19:48 (hora de Lima, UTC−5); cada bloque lleva su marca de tiempo. Las capturas de Kafka UI y Eureka se tomaron en la misma sesión y muestran sus propias marcas de tiempo.

01

Introducción y motivación

S8 dejó dos preguntas abiertas. La primera: Kafka entrega **al menos una vez**, así que un consumidor puede recibir el mismo mensaje dos veces (por ejemplo, si procesa el mensaje y se reinicia antes de confirmar el offset). ¿Qué le pasa a `veterinaria-pago-ms` cuando le llega otra vez un `cita.creada` que ya procesó? La segunda: la pasarela de pagos estaba simulada y **siempre** validaba. ¿Qué pasa con la cita si el pago falla después de que la cita ya se registró?

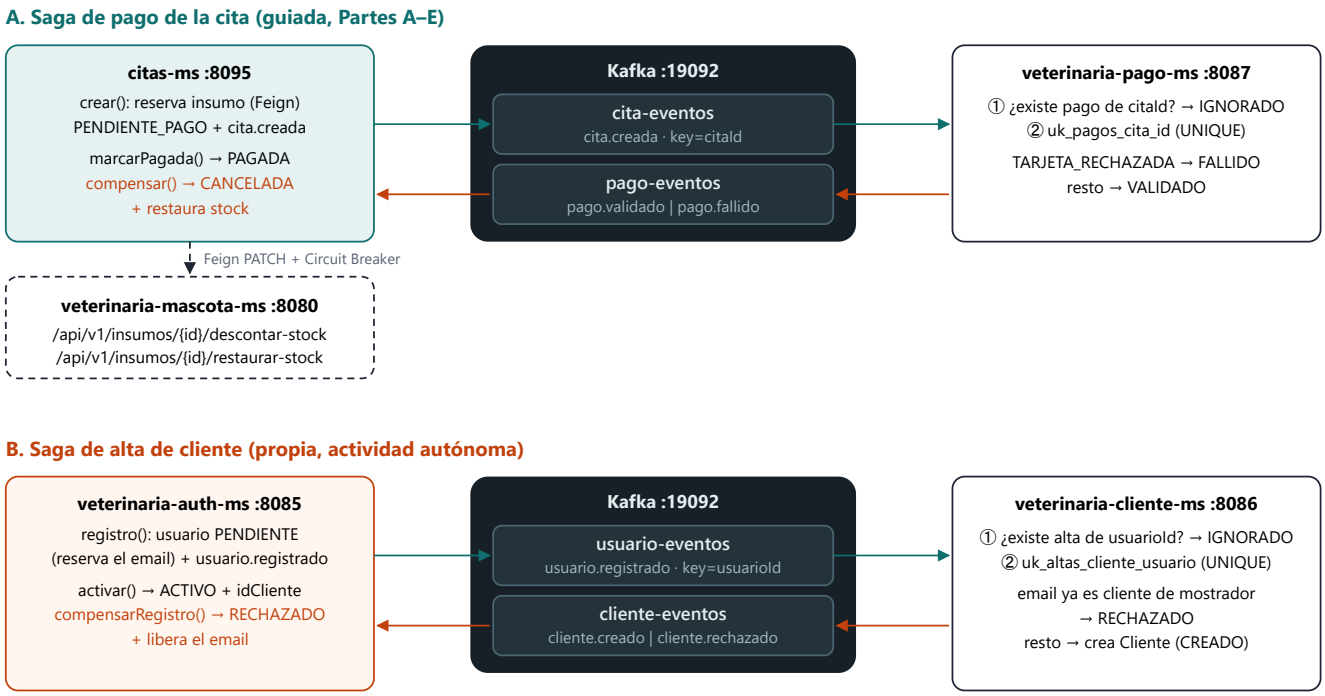
El caso Knight Capital (SEC, 2013) muestra el costo de no tener la primera respuesta: un servidor con una bandera antigua volvía a ejecutar la misma acción con cada señal que llegaba, sin ningún guardia que dijera "esto ya lo procesé", y en 45 minutos acumuló una pérdida de unos 440 millones de dólares. En esta sesión se provoca a propósito el mismo mecanismo, a escala mínima, contra el propio `veterinaria-pago-ms`, y luego se cierra con dos piezas: **idempotencia** (un paso repetido no tiene efecto doble) y **compensación** (un paso ya confirmado se deshace con una nueva transición de estado cuando un paso posterior falla). Juntas forman una **Saga coreografiada**.

Garantía	Quién la da	En este sistema
El mensaje llega, en orden dentro de su partición	Kafka	`acks=all`, key = `citaId` / `usuarioId`
El mensaje nunca llega duplicado	Nadie (al menos una vez)	Se asume que sí puede llegar duplicado
Procesar un duplicado no causa efecto doble	El consumidor	Revisión explícita + restricción `UNIQUE` (sección 3 y 8)
Un paso que falla se deshace en los anteriores	La Saga	`pago.fallido` y `cliente.rechazado` (secciones 4 y 8)
El evento no se pierde si el servicio cae entre guardar y publicar	—	No resuelto (requiere Outbox); limitación conocida

02

Arquitectura: dos sagas coreografiadas

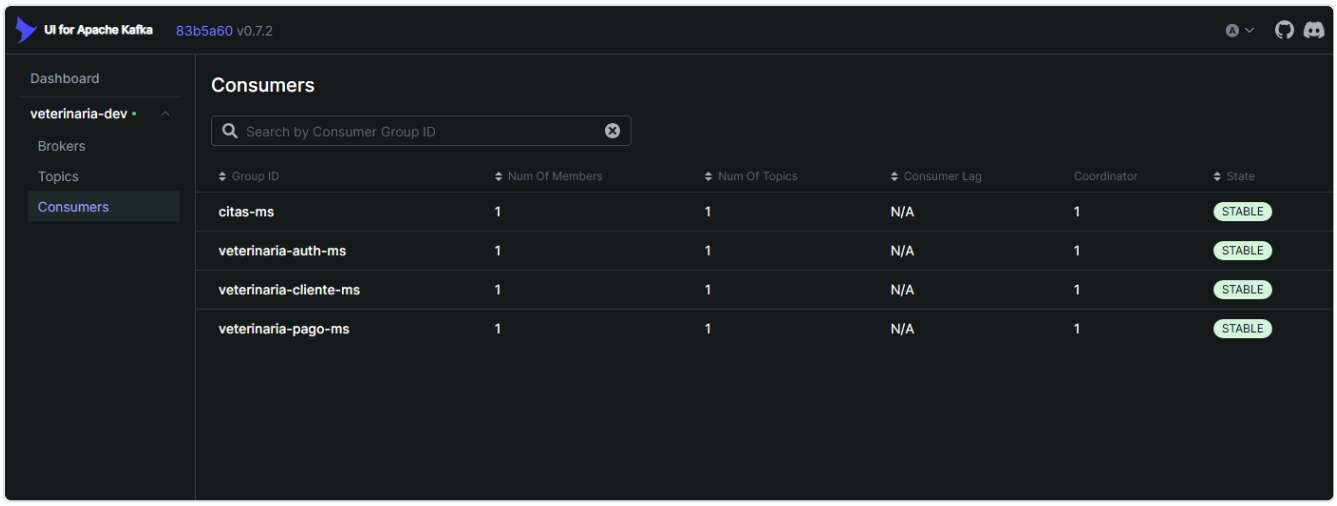
No hay coordinador central. Cada servicio escucha los eventos que le interesan, ejecuta su transacción local y publica lo que hizo. La saga de pago (guiada) cruza `citas-ms` y `veterinaria-pago-ms`; la saga de alta de cliente (propia) cruza `veterinaria-auth-ms` y `veterinaria-cliente-ms`. En ambas el primer consumidor es idempotente y el servicio que inició el proceso tiene un paso de confirmación y otro de compensación con la **misma guardia**.



Reglas comunes de ambas sagas

- El evento se publica solo después del commit de la transacción local (sin eventos fantasma).
- Primer consumidor idempotente con dos capas: revisión explícita del resultado + restricción UNIQUE real en PostgreSQL.
- Confirmación y compensación comparten la misma guardia (solo actúan desde PENDIENTE_*) y nunca borran: dejan un estado final auditable.
- Cada servicio tiene su propia copia de los contratos de eventos y su propio consumer group.

Figura 1. Las dos sagas del sistema. Flechas verdes: camino de ida; flechas naranjas: respuesta (éxito o fallo). Línea punteada: llamada síncrona Feign.



Kafka UI → Consumers: los cuatro consumer groups activos (STABLE), uno por servicio consumidor; `veterinaria-auth-ms` y `veterinaria-cliente-ms` son nuevos en S9.

03

Bloque 1 — El bug de duplicados y la idempotencia de veterinaria-pago-ms

3.1 Punto de partida

Con todo el sistema arriba (Config, Eureka, Gateway, auth, mascota, cliente, citas, pago, Kafka y seis PostgreSQL) se crea una cita con el token del cliente y se confirma que el flujo de S8 sigue funcionando:

```
$ curl -X POST $GW/api/v1/citas -H "Authorization: Bearer <TOKEN CLIENTE>" -d
'{"mascotaId":1,"motivo":"Control anual (S9 punto de partida)","fechaHora":"2026-10-
22T10:00:00","metodoPago":"YAPE","monto":80.00}'   # 2026-10-15 19:27:00
HTTP/1.1 201
{"id":8,"idCliente":1,"mascotaId":1,"mascotaNombre":"Firulais",...,"estado":"PENDIENTE_PAGO","monto":
80.00,"metodoPago":"YAPE",...}
$ curl $GW/api/v1/citas/8   # 2026-10-15 19:27:07
{"id":8,...,"estado":"PAGADA","monto":80.00,"metodoPago":"YAPE",...}
```

3.2 Reproducción del bug

El código de S8 ya buscaba el pago por `cita_id`, pero al encontrarlo **volvía a publicar** `pago.validado` (otro efecto repetido). Para reproducir el bug de la guía tal cual, se compiló una versión de `PagoServiceImpl.procesar()` sin ninguna revisión (solo el `save`) y se reenvió a `cita-eventos` el mismo mensaje de la cita 8, con la misma key y el mismo valor, leído del propio tópico:

```
$ printf '8|
{"tipoEvento":"cita.creada","citaId":8,"idCliente":1,"mascotaId":1,"monto":80.00,"metodoPago":"YAPE",
"origen":"citas-ms","timestamp":1792110423393}\n' \
   | kafka-console-producer.sh --topic cita-eventos --property parse.key=true --property
key.separator='|'   # 19:29:20

### log de veterinaria-pago-ms (version sin idempotencia)
2026-10-15 19:29:20.662 INFO  CitaEventosConsumer - component=cita-consumer eventType=cita.creada
citaId=8 status=RECIBIDO partition=0 offset=5
2026-10-15 19:29:20.705 WARN  org.hibernate.orm.jdbc.error - HHH000247: ErrorCode: 0, SQLState: 23505
2026-10-15 19:29:20.705 WARN  org.hibernate.orm.jdbc.error - ERROR: duplicate key value violates
unique constraint "uk_pagos_cita_id"
  Detail: Key (cita_id)=(8) already exists.
2026-10-15 19:29:20.787 INFO  KafkaMessageListenerContainer - Record in retry and not yet recovered
2026-10-15 19:29:21.277 INFO  CitaEventosConsumer - component=cita-consumer eventType=cita.creada
citaId=8 status=RECIBIDO partition=0 offset=5
2026-10-15 19:29:21.292 WARN  org.hibernate.orm.jdbc.error - ERROR: duplicate key value violates
unique constraint "uk_pagos_cita_id"
   ... (el mismo offset se reintenta 10 veces, cada ~0,5 s) ...
2026-10-15 19:29:25.408 WARN  org.hibernate.orm.jdbc.error - ERROR: duplicate key value violates
unique constraint "uk_pagos_cita_id"
2026-10-15 19:29:25.427 ERROR o.s.k.listener.DefaultErrorHandler - Backoff
FixedBackOffExecution[interval=0, currentAttempts=10, maxAttempts=9] exhausted for cita-eventos-0@5
org.springframework.kafka.listener.ListenerExecutionFailedException: Listener method
'...CitaEventosConsumer.consumir(...)' threw exception
Caused by: org.springframework.dao.DataIntegrityViolationException: could not execute statement
[ERROR: duplicate key value violates unique constraint "uk_pagos_cita_id"
```

```
$ psql veterinaria_pago_db -c "SELECT COUNT(*) FROM pagos WHERE cita_id = 8;"
 pagos_cita_8
--------------
            1
```

Lo que muestra el log: el consumidor recibe el mismo offset **10 veces** (1 intento + 9 reintentos del `DefaultErrorHandler`), cada vez `save()` intenta crear un segundo pago y la restricción `uk_pagos_cita_id` lo rechaza. Nada en el código esperaba esa excepción. La base de datos se protegió sola (sigue habiendo un pago), pero el servicio gastó 10 intentos, llenó el log de errores y al final descartó el mensaje como fallido.

3.3 Idempotencia en dos capas

```
// PagoServiceImpl.java (veterinaria-pago-ms)
@Override
public void procesar(CitaCreadaEvento evento) {
    Pago existente = pagoRepository.findByCitaId(evento.getCitaId()).orElse(null);
    if (existente != null) {                                    // capa 1: revision explicita
        log.warn("component=pago-service eventType={} citaId={} status=IGNORADO motivo=\"pago ya
procesado (evento duplicado)\" pagoId={} estado={}", ...);
        return;                                                 // no guarda ni vuelve a publicar
    }
    boolean rechazado = METODO_RECHAZADO.equalsIgnoreCase(evento.getMetodoPago());
    Pago pago;
    try {
        pago = pagoRepository.save(Pago.builder()...estado(rechazado ? EstadoPago.FALLIDO :
EstadoPago.VALIDADO)...build());
    } catch (DataIntegrityViolationException ex) {              // capa 2: uk_pagos_cita_id
        log.warn("component=pago-service ... status=IGNORADO motivo=\"pago ya procesado (condicion de
carrera, uk_pagos_cita_id)\"", ...);
        return;
    }
    log.info("component=pago-service ... status=PROCESADO pagoId={} estado={} ...", ...);
    pagoEventosProducer.publicarResultado(pago, rechazado ? MOTIVO_RECHAZO : null);
}
```

- **Capa 1 — revisión explícita** (`findByCitaId`): resuelve el caso normal (una reentrega de Kafka detrás de otra) sin intentar un INSERT que se sabe que fallará. La idempotencia se verifica contra el **resultado** (¿ya existe el pago de esta cita?), no contra "ya vi este mensaje".
- **Capa 2 — restricción UNIQUE** (`uk_pagos_cita_id`, ya existente desde S8): es la garantía real si dos entregas pasan la revisión a la vez (dos instancias del servicio, por ejemplo). Una revisión en memoria no evita esa carrera; la base de datos sí.
- `procesar()` ya no es `@Transactional` a propósito: `save()` confirma en su propia transacción, así la violación UNIQUE se captura sin dejar una transacción externa marcada como *rollback-only* (ver sección 9), y el evento se publica solo después de ese commit.

3.4 El duplicado ya no rompe nada

Se reinicia `veterinaria-pago-ms` con el código corregido y se repite exactamente el mismo reenvío:

```
$ reenvio de cita.creada citaId=8 (misma key y valor), pago-ms con idempotencia   # 2026-10-15
19:33:29
2026-10-15 19:33:29.764 INFO  CitaEventosConsumer - component=cita-consumer eventType=cita.creada
```

```
citaId=8 status=RECIBIDO partition=0 offset=7
2026-10-15 19:33:31.534 WARN  PagoServiceImpl - component=pago-service eventType=cita.creada citaId=8
status=IGNORADO motivo="pago ya procesado (evento duplicado)" pagoId=5 estado=VALIDADO
$ psql veterinaria_pago_db -c "SELECT COUNT(*) AS pagos_cita_8 FROM pagos WHERE cita_id = 8;"
 pagos_cita_8
--------------
            1
```

Resultado: un solo intento, ninguna excepción, ningún segundo pago y ningún `pago.validado` repetido.

04

Bloque 2 — Fallo de pago y compensación de la cita

3.5 Estado FALLIDO y rechazo simulado

`EstadoPago` gana el valor `FALLIDO`. El valor reservado `metodoPago: "TARJETA_RECHAZADA"` hace que la pasarela simulada rechace siempre, de forma determinista, para que la prueba sea repetible. El productor elige el tipo de evento según el estado y agrega el motivo:

```
public enum EstadoPago { VALIDADO, FALLIDO }

// PagoEventosProducer.publicarResultado(pago, motivo)
boolean fallido = pago.getEstado() == EstadoPago.FALLIDO;
PagoValidadoEvento evento = PagoValidadoEvento.builder()
        .tipoEvento(fallido ? PagoValidadoEvento.TIPO_FALLIDO : PagoValidadoEvento.TIPO)   //
"pago.fallido" | "pago.validado"
        .citaId(pago.getCitaId()).pagoId(pago.getId()).monto(pago.getMonto())
        .metodoPago(pago.getMetodoPago()).estado(pago.getEstado().name())
        .motivo(motivo).origen(origen).timestamp(System.currentTimeMillis())
        .build();
```

En `citas-ms`, `CitaRequest` acepta el nuevo valor (`@Pattern(regexp = "TARJETA|YAPE|EFECTIVO|TARJETA_RECHAZADA")`). En un sistema real la decisión vendría de la respuesta de la pasarela, nunca de un valor que declara el cliente.

3.6 Consumir pago.fallido y compensar

```
// PagoEventosConsumer.java (citas-ms)
switch (evento.getTipoEvento()) {
    case PagoValidadoEvento.TIPO -> citaService.marcarPagada(evento.getCitaId(), evento.getPagoId());
    case PagoValidadoEvento.TIPO_FALLIDO -> citaService.compensar(evento.getCitaId(),
evento.getPagoId(), evento.getMotivo());
    case null, default -> log.info("component=pago-consumer eventType={} citaId={} status=IGNORADO
motivo=tipo_no_soportado", ...);
}

// CitaService.java (citas-ms)
@Transactional
public void compensar(Long citaId, Long pagoId, String motivo) {
    Cita cita = citaRepository.findById(citaId).orElse(null);
    if (cita == null || !ESTADO_PENDIENTE_PAGO.equals(cita.getEstado())) {      // misma guardia que
marcarPagada
        log.warn("component=cita-service eventType=pago.fallido citaId={} status=IGNORADO motivo=\"la
cita no existe o ya no esta pendiente de pago\" estadoActual={}", ...);
        return;
    }
    if (cita.getInsumoId() != null && cita.getCantidadInsumo() != null) {
        insumoClientService.restaurarStock(cita.getInsumoId(), cita.getCantidadInsumo());   // Parte
E
    }
    cita.setEstado(ESTADO_CANCELADA);                                          // no se borra:
rastro auditable
    citaRepository.save(cita);
```

```
    log.info("component=cita-service eventType=pago.fallido citaId={} status=COMPENSADA pagoId={}
estado={}->{} motivoPago=\"{}\"", ...);
}
```

Las constantes de los `case` son `public static final String` en la clase del contrato: un `case` con una constante no final no compila (ver error frecuente de la guía). `case null` evita un `NullPointerException` si llega un evento sin `tipoEvento`.

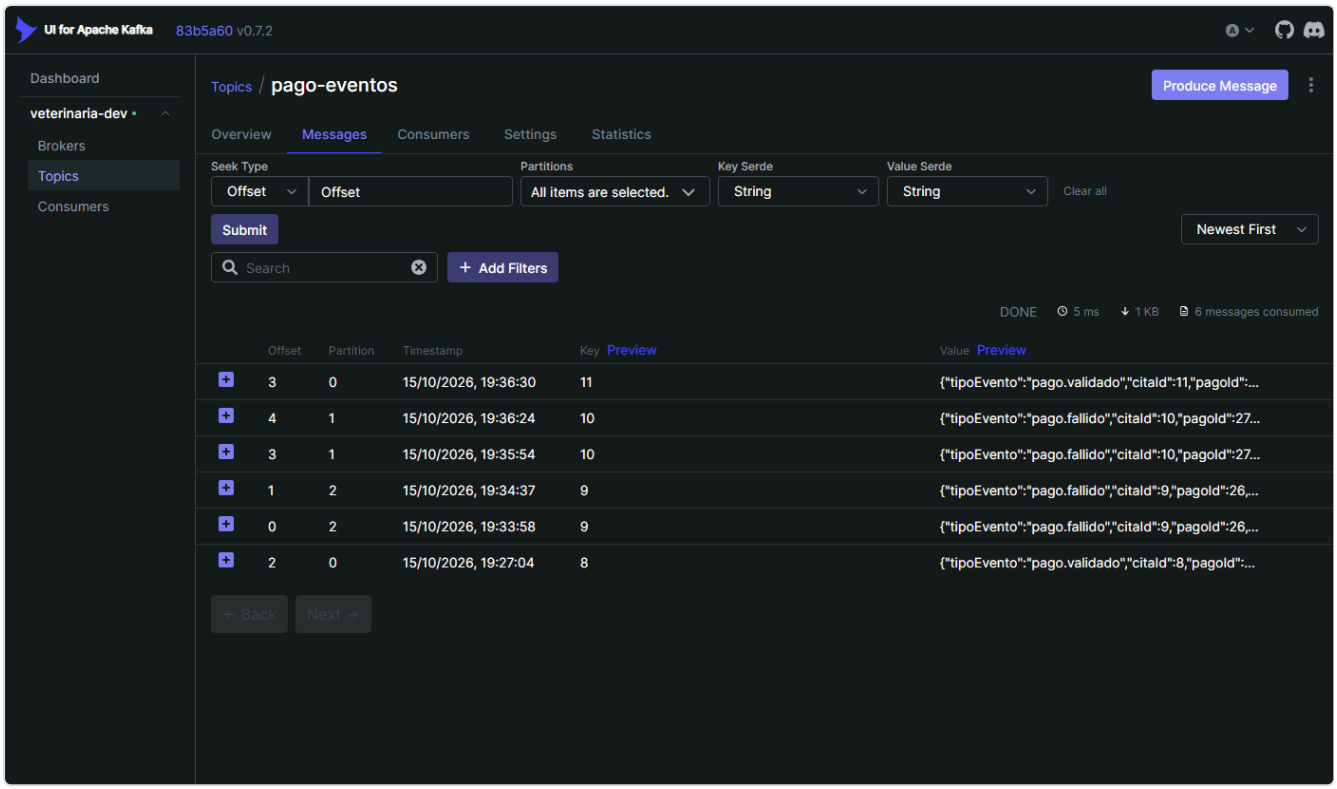
3.7 Prueba de punta a punta

```
$ curl -X POST $GW/api/v1/citas -H "Authorization: Bearer <TOKEN CLIENTE>" -d
'{"mascotaId":2,"motivo":"Desparasitacion (S9 pago rechazado)","fechaHora":"2026-10-
23T09:00:00","metodoPago":"TARJETA_RECHAZADA","monto":60.00}'   # 2026-10-15 19:33:58
HTTP/1.1 201
{"id":9,...,"mascotaNombre":"Michi",...,"estado":"PENDIENTE_PAGO","monto":60.00,"metodoPago":"TARJETA
_RECHAZADA",...}

### veterinaria-pago-ms
19:33:58.620 INFO  CitaEventosConsumer - component=cita-consumer eventType=cita.creada citaId=9
status=RECIBIDO partition=2 offset=0
19:33:58.817 INFO  PagoServiceImpl - component=pago-service eventType=cita.creada citaId=9
status=PROCESADO pagoId=26 estado=FALLIDO monto=60.00 metodoPago=TARJETA_RECHAZADA
19:33:59.088 INFO  PagoEventosProducer - component=pago-producer eventType=pago.fallido citaId=9
status=PUBLICADO topic=pago-eventos partition=2 offset=0
### citas-ms
19:33:58.616 INFO  CitaEventosProducer - component=cita-producer eventType=cita.creada citaId=9
status=PUBLICADO topic=cita-eventos partition=2 offset=0
19:33:59.081 INFO  PagoEventosConsumer - component=pago-consumer eventType=pago.fallido citaId=9
status=RECIBIDO partition=2 offset=0
19:33:59.093 INFO  CitaService - component=cita-service eventType=pago.fallido citaId=9
status=COMPENSADA pagoId=26 estado=PENDIENTE_PAGO->CANCELADA motivoPago="Tarjeta rechazada por la
pasarela (simulada)"

$ curl $GW/api/v1/pagos  (token CLIENTE, filtrado citaId=9)   # 19:34:16
{"id": 26, "citaId": 9, "idCliente": 1, "monto": 60.0, "metodoPago": "TARJETA_RECHAZADA", "estado":
"FALLIDO", "fechaPago": "2026-10-15T19:33:58.634633"}
$ curl $GW/api/v1/citas/9   # 19:34:16
{"id":9,...,"estado":"CANCELADA","monto":60.00,"metodoPago":"TARJETA_RECHAZADA",...}
$ psql veterinaria_citas_db                          $ psql veterinaria_pago_db
 id |  estado   |    metodo_pago                      id | cita_id |    metodo_pago    |  estado
----+-----------+------------------                  ----+---------+-------------------+----------
  8 | PAGADA    | YAPE                                 5 |       8 | YAPE              | VALIDADO
  9 | CANCELADA | TARJETA_RECHAZADA                   26 |       9 | TARJETA_RECHAZADA | FALLIDO
```

La saga completa (publicar, consumir, rechazar, publicar el fallo y compensar) tardó 477 ms (19:33:58.616 → 19:33:59.093). La cita nunca llegó a `PAGADA`. El salto de id del pago (5 → 26) se explica en la sección 9.



Kafka UI → `pago-eventos`: `pago.validado` de las citas 8 y 11, `pago.fallido` de las citas 9 y 10. Los mensajes de offset 1 (partición 2) y 4 (partición 1) son los reenvíos manuales de las pruebas de compensación duplicada.

05

Bloque 3 — La compensación también es idempotente (3.8)

Se lee del tópico el `pago.fallido` de la cita 9 y se vuelve a publicar con la misma key y el mismo valor, simulando una reentrega de Kafka:

```
$ reenvio de pago.fallido citaId=9 (misma key y valor)   # 2026-10-15 19:34:35
9|
{"tipoEvento":"pago.fallido","citaId":9,"pagoId":26,"monto":60.00,"metodoPago":"TARJETA_RECHAZADA","estado":"FALLIDO","motivo":"Tarjeta rechazada por la pasarela (simulada)","origen":"veterinaria-pago-ms","timestamp":1792110838820}

### citas-ms
2026-10-15 19:34:37.604 INFO  PagoEventosConsumer - component=pago-consumer eventType=pago.fallido citaId=9 status=RECIBIDO partition=2 offset=1
2026-10-15 19:34:37.616 WARN  CitaService - component=cita-service eventType=pago.fallido citaId=9 status=IGNORADO motivo="la cita no existe o ya no esta pendiente de pago" estadoActual=CANCELADA
$ psql veterinaria_citas_db -c "SELECT id, estado FROM citas WHERE id = 9;"
 id |  estado
----+-----------
  9 | CANCELADA
```

No se escribió nada nuevo para este caso: la guardia `estado == PENDIENTE_PAGO` que ya protegía `marcarPagada` desde S8 protege también a `compensar`. Con la Parte E activa la guardia importa todavía más: sin ella, un `pago.fallido` repetido devolvería el stock dos veces. La sección 7 lo prueba con la cita 10: el reenvío se ignora y el stock no cambia.

La misma regla en el otro sentido: un `cita.creada` repetido de una cita cuyo pago ya está `FALLIDO` también se ignora en `veterinaria-pago-ms` (capa 1) y no vuelve a publicar `pago.fallido`:

```
$ reenvio de cita.creada citaId=9   # 2026-10-15 19:47:25
19:47:27.732 INFO  CitaEventosConsumer - component=cita-consumer eventType=cita.creada citaId=9 status=RECIBIDO partition=2 offset=1
19:47:27.750 WARN  PagoServiceImpl - component=pago-service eventType=cita.creada citaId=9 status=IGNORADO motivo="pago ya procesado (evento duplicado)" pagoId=26 estado=FALLIDO
$ psql veterinaria_pago_db -c "SELECT COUNT(*) FROM pagos WHERE cita_id=9;"  ->  1
```

06

Parte D — Estados, transiciones y compensaciones de Cita (3.9)

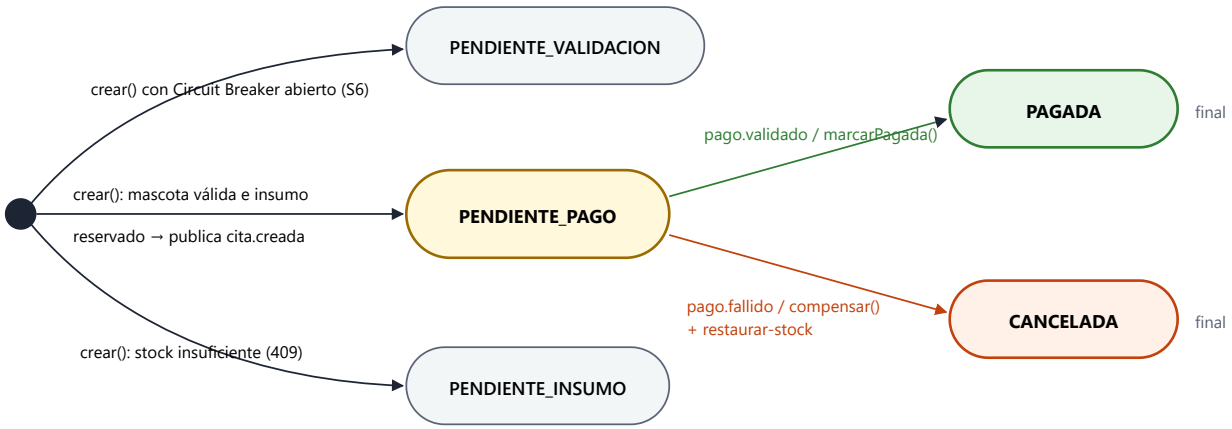


Figura 2. Ciclo de vida de Cita. Las citas en `PENDIENTE_VALIDACION` o `PENDIENTE_INSUMO` no publican evento: no se cobra una atención que no se pudo validar o abastecer.

Paso de la saga	Servicio	Si falla	Compensación
1. Registrar la cita y reservar el insumo	citas-ms → veterinaria-mascota-ms (Feign)	Mascota no validable o stock insuficiente: no se publica nada	No aplica: no se confirmó ningún paso remoto
2. Validar el pago	veterinaria-pago-ms	La pasarela simulada rechaza (`TARJETA_RECHAZADA`)	Guarda el pago `FALLIDO` y publica `pago.fallido` en vez de `pago.validado`
3. Confirmar la cita	citas-ms	— (último paso)	Al recibir `pago.fallido` : `PENDIENTE_PAGO → CANCELADA` y devolver el insumo reservado en el paso 1

07

Parte E (bono) — Reservar y devolver stock de insumos

En la veterinaria el equivalente al stock del catálogo es el **inventario de insumos clínicos** (vacunas, antiparasitarios) que consume una atención. Se agregó a `veterinaria-mascota-ms`, el servicio que `citas-ms` ya consulta por Feign:

- **veterinaria-mascota-ms**: migración `V2__create_insumos.sql` (tabla `insumos` con `CHECK (stock >= 0)` y tres insumos de prueba), `InsumoService.descontarStock/restaurarStock` con `@Transactional` (leer, comparar y guardar en una sola transacción), `StockInsuficienteException` → **409**, y `PATCH /api/v1/insumos/{id}/descontar-stock|restaurar-stock?cantidad=n`.
- **citas-ms**: dependencia `feign-hc5` (el cliente HTTP por defecto del JDK no soporta PATCH), `InsumoClient` + `InsumoClientService` con el mismo Circuit Breaker `mascotaService` de S6, migración `V5__add_insumo_to_citas.sql` (`insumo_id`, `cantidad_insumo`) para saber exactamente qué devolver.
- **Gateway**: ruta `/api/v1/insumos/**`; lectura CLIENTE o ADMIN, escritura solo ADMIN (citas-ms llama directo por Eureka).

```
// InsumoClientService.java (citas-ms)
@CircuitBreaker(name = "mascotaService", fallbackMethod = "fallbackDescontar")
public boolean descontarStock(Long insumoId, Integer cantidad) {
insumoClient.descontarStock(insumoId, cantidad); ...; return true; }
private boolean fallbackDescontar(Long insumoId, Integer cantidad, Throwable ex) { log.warn("...
status=RESERVA_FALLIDA ..."); return false; }

@CircuitBreaker(name = "mascotaService", fallbackMethod = "fallbackRestaurar")
public void restaurarStock(Long insumoId, Integer cantidad) { insumoClient.restaurarStock(insumoId,
cantidad); ... }
// No relanza: la cita queda CANCELADA igual; el stock queda pendiente de correccion manual.
private void fallbackRestaurar(Long insumoId, Integer cantidad, Throwable ex) { log.error("...
status=RESTAURACION_FALLIDA ... accion=requiere_correccion_manual"); }
```

Tabla 5 reproducida: stock en cada momento

Para poder fotografiar el momento intermedio se detuvo `veterinaria-pago-ms` antes de crear la cita: así la saga queda en `PENDIENTE_PAGO` (el evento espera en el broker) hasta que el servicio vuelve.

```
### Momento 1: antes de crear la cita   # 2026-10-15 19:34:59
$ curl $GW/api/v1/insumos/1
{"id":1,"nombre":"Vacuna antirrabica","tipo":"VACUNA","stock":50}
### veterinaria-pago-ms detenido   # 19:35:10; health 8087 -> 000
### Momento 2: POST /api/v1/citas con insumoId=1, cantidadInsumo=2, TARJETA_RECHAZADA   # 19:35:12
HTTP/1.1 201
{"id":10,...,"estado":"PENDIENTE_PAGO","monto":95.00,"metodoPago":"TARJETA_RECHAZADA","insumoId":1,"c
antidadInsumo":2}
$ curl $GW/api/v1/insumos/1   # 19:35:13
{"id":1,"nombre":"Vacuna antirrabica","tipo":"VACUNA","stock":48}
### Momento 3: pago-ms reiniciado; citas-ms consume pago.fallido y compensa   # 19:35:56
$ curl $GW/api/v1/insumos/1
{"id":1,"nombre":"Vacuna antirrabica","tipo":"VACUNA","stock":50}
$ curl $GW/api/v1/citas/10
{"id":10,...,"estado":"CANCELADA",...,"insumoId":1,"cantidadInsumo":2}
```

```
### veterinaria-mascota-ms
19:35:13.299 INFO  InsumoService - component=insumo-service insumoId=1 cantidad=2
status=STOCK_DESCONTADO stock=50->48
19:35:55.022 INFO  InsumoService - component=insumo-service insumoId=1 cantidad=2
status=STOCK_RESTAURADO stock=48->50
### citas-ms
19:35:13.445 INFO  InsumoClientService - component=insumo-client insumoId=1 cantidad=2
status=STOCK_RESERVADO
19:35:13.501 INFO  CitaEventosProducer - component=cita-producer eventType=cita.creada citaId=10
status=PUBLICADO topic=cita-eventos partition=1 offset=3
19:35:54.951 INFO  PagoEventosConsumer - component=pago-consumer eventType=pago.fallido citaId=10
status=RECIBIDO partition=1 offset=3
19:35:55.058 INFO  InsumoClientService - component=insumo-client insumoId=1 cantidad=2
status=STOCK_RESTAURADO
19:35:55.058 INFO  CitaService - component=cita-service eventType=pago.fallido citaId=10
status=COMPENSADA pagoId=27 estado=PENDIENTE_PAGO->CANCELADA ...
```

Momento	Hora	Stock insumo 1
Antes de crear la cita	19:34:59	50
Justo después de `POST /api/v1/citas` (TARJETA_RECHAZADA, cantidad 2)	19:35:13	48
Después de que citas-ms consume `pago.fallido` y compensa	19:35:56	50

Casos adicionales

```
### a) reenvio de pago.fallido citaId=10 (compensacion duplicada)   # 19:36:22
19:36:24.980 WARN  CitaService - component=cita-service eventType=pago.fallido citaId=10
status=IGNORADO motivo="la cita no existe o ya no esta pendiente de pago" estadoActual=CANCELADA
$ curl $GW/api/v1/insumos/1  ->  "stock":50          # no se devolvio dos veces
### b) pago normal (YAPE) con insumoId=2: el stock baja y se queda bajo   # 19:36:30
stock antes 30 -> cita 11 PENDIENTE_PAGO -> pago.validado -> CITA_PAGADA -> stock despues 29
### c) stock insuficiente: insumoId=3 (stock 3), cantidadInsumo=5   # 19:36:35
veterinaria-mascota-ms: component=insumo-service insumoId=3 cantidad=5 stock=3
status=STOCK_INSUFICIENTE
citas-ms:              component=insumo-client insumoId=3 cantidad=5 status=RESERVA_FALLIDA motivo="
[409 ] during [PATCH] to [http://veterinaria-mascota-ms/api/v1/insumos/3/descontar-stock?cantidad=5]
..."
HTTP/1.1 201  {"id":12,...,"estado":"PENDIENTE_INSUMO","insumoId":3,"cantidadInsumo":null}   # no
publica cita.creada; stock sigue en 3
```

Una cita sin el insumo reservado queda en `PENDIENTE_INSUMO` y guarda `cantidad_insumo = null`, así una compensación nunca podría devolver stock que no se reservó (el error frecuente de 3.12).

08

Bloque 4 — Proceso propio: saga de alta de cliente

El proceso y por qué necesita una saga

Hasta S8, `POST /api/v1/auth/registro` creaba la cuenta en `veterinaria-auth-ms` pero nadie creaba el perfil en `veterinaria-cliente-ms`: la cuenta quedaba sin `idCliente` en el JWT y no podía usar `/citas` ni `/clientes/me`. El alta de un cliente cruza dos servicios con bases de datos separadas, así que se resolvió con el mismo patrón de la sesión:

1. **auth-ms** guarda la cuenta `PENDIENTE` y deshabilitada —con eso **reserva el email**— y publica `usuario.registrado` (responde **202 Accepted**).
2. **cliente-ms** consume el evento de forma idempotente y crea el perfil, o lo rechaza si el email ya pertenece a un cliente registrado en mostrador (condición de fallo real: la recepción debe verificar a la persona antes de vincular la cuenta). Publica `cliente.creado` o `cliente.rechazado`.
3. **auth-ms** confirma (`PENDIENTE → ACTIVO`, guarda `idCliente`, habilita el login) o **compensa** (`PENDIENTE → RECHAZADO`, deshabilitada y con el email **liberado**).

Idempotencia del primer consumidor: revisión + UNIQUE real

```
-- V2__create_altas_cliente.sql (veterinaria-cliente-ms)
CREATE TABLE IF NOT EXISTS altas_cliente (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    usuario_id BIGINT NOT NULL, email VARCHAR(150) NOT NULL,
    resultado VARCHAR(20) NOT NULL,        -- CREADO | RECHAZADO
    motivo VARCHAR(255), cliente_id BIGINT, fecha_proceso TIMESTAMP NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uk_altas_cliente_usuario UNIQUE (usuario_id)
);
```

```
// AltaClienteService.procesar() — fuera de transaccion
AltaCliente previa = altaClienteRepository.findByUsuarioId(evento.getUsuarioId()).orElse(null);
if (previa != null) { log.warn("... status=IGNORADO motivo=\"alta ya procesada (evento duplicado)\"
resultadoPrevio={}", ...); return; }
AltaCliente alta;
try {
    alta = altaClienteTx.registrar(evento);         // @Transactional: Cliente + AltaCliente juntos,
o nada
} catch (DataIntegrityViolationException ex) {
    if (altaClienteRepository.findByUsuarioId(evento.getUsuarioId()).isPresent()) { log.warn("...
condicion de carrera, uk_altas_cliente_usuario ..."); return; }
    throw ex;
}
clienteEventosProducer.publicarResultado(alta);       // cliente.creado | cliente.rechazado, despues
del commit

// AltaClienteTx.registrar() — condicion de fallo de negocio
if (clienteRepository.existsByEmail(evento.getEmail())) {
    return
altaClienteRepository.saveAndFlush(alta.resultado(RECHAZADO).motivo(MOTIVO_EMAIL_EN_USO).build());
}
```

El registro de idempotencia guarda también los rechazos (igual que el pago `FALLIDO`): un `usuario.registrado` repetido de una cuenta rechazada no se vuelve a evaluar ni vuelve a publicar `cliente.rechazado`. La transacción local va en un bean aparte (`AltaClienteTx`) para capturar la violación UNIQUE fuera de la transacción que la produjo.

Compensación con la misma guardia, y liberación del recurso

```
-- V2__estado_registro_usuarios.sql (veterinaria-auth-ms)
ALTER TABLE usuarios ADD COLUMN estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO';
ALTER TABLE usuarios ADD COLUMN fecha_registro TIMESTAMP NOT NULL DEFAULT now();
ALTER TABLE usuarios DROP CONSTRAINT uk_usuarios_email;
CREATE UNIQUE INDEX uk_usuarios_email_vigente ON usuarios (email) WHERE estado <> 'RECHAZADO';
```

```
// AuthService.java (veterinaria-auth-ms)
@Transactional
public void compensarRegistro(Long usuarioId, String motivo) {
    Usuario usuario = usuarioRepository.findById(usuarioId).orElse(null);
    if (usuario == null || usuario.getEstado() != EstadoUsuario.PENDIENTE) {        // misma guardia
que activar()
        log.warn("component=auth-service eventType=cliente.rechazado usuarioId={} status=IGNORADO
motivo=\"la cuenta no existe o ya no esta pendiente\" estadoActual={}", ...);
        return;
    }
    usuario.setEstado(EstadoUsuario.RECHAZADO);   // no se borra: rastro auditable
    usuario.setHabilitado(false);                 // no puede iniciar sesion
    usuarioRepository.save(usuario);              // el indice parcial excluye RECHAZADO: email
liberado
    log.info("... status=COMPENSADA estado=PENDIENTE->RECHAZADO email={} recurso=email_liberado
motivoRechazo=\"{}\"", ...);
}
```

El email es el recurso que reserva el paso 1. Si la compensación solo cambiara el estado, la fila rechazada seguiría ocupando el email para siempre. Con el índice único **parcial** la fila queda como rastro y el email vuelve a estar disponible. Login y `UserDetailsService` buscan ahora la cuenta *vigente* (`findByEmailAndEstadoNot(email, RECHAZADO)`).

Evento	Tópico / key	Campos
`usuario.registrado`	usuario-eventos / usuarioId	tipoEvento, usuarioId, email, nombres, apellidos, telefono, direccion, origen, timestamp
`cliente.creado`	cliente-eventos / usuarioId	tipoEvento, usuarioId, clienteId, email, origen, timestamp
`cliente.rechazado` (compensación)	cliente-eventos / usuarioId	tipoEvento, usuarioId, email, **motivo**, origen, timestamp

Caso 1 — Camino de éxito

```
$ curl -X POST $GW/api/v1/auth/registro -d
'{"email":"maria.lopez@gmail.com","password":"maria123","nombres":"Maria","apellidos":"Lopez
Huaman","telefono":"951222333","direccion":"Jr. Puno 210, Juliaca"}'   # 2026-10-15 19:36:58
HTTP/1.1 202
{"usuarioId":5,"email":"maria.lopez@gmail.com","estado":"PENDIENTE","idCliente":null,"fechaRegistro":
"2026-10-15T19:36:58.5841125"}
### auth-ms
19:36:58.705 INFO  AuthService - component=auth-service usuarioId=5 email=maria.lopez@gmail.com
status=REGISTRO_PENDIENTE estado=PENDIENTE
19:36:59.059 INFO  UsuarioEventosProducer - component=usuario-producer eventType=usuario.registrado
usuarioId=5 status=PUBLICADO topic=usuario-eventos partition=0 offset=0
### cliente-ms
19:36:59.098 INFO  UsuarioEventosConsumer - component=usuario-consumer eventType=usuario.registrado
usuarioId=5 status=RECIBIDO partition=0 offset=0
19:37:01.310 INFO  AltaClienteService - component=alta-cliente-service eventType=usuario.registrado
usuarioId=5 status=PROCESADO resultado=CREADO clienteId=3 email=maria.lopez@gmail.com
19:37:01.596 INFO  ClienteEventosProducer - component=cliente-producer eventType=cliente.creado
usuarioId=5 status=PUBLICADO topic=cliente-eventos partition=0 offset=0
### auth-ms
19:37:01.608 INFO  ClienteEventosConsumer - component=cliente-consumer eventType=cliente.creado
usuarioId=5 status=RECIBIDO partition=0 offset=0
19:37:01.667 INFO  AuthService - component=auth-service eventType=cliente.creado usuarioId=5
status=CUENTA_ACTIVADA idCliente=3 estado=PENDIENTE->ACTIVO
$ curl $GW/api/v1/auth/registro/5   # 19:37:17
{"usuarioId":5,"email":"maria.lopez@gmail.com","estado":"ACTIVO","idCliente":3,...}
$ login maria.lopez@gmail.com -> claims: {"email": "maria.lopez@gmail.com", "idCliente": 3,
"realm_access": {"roles": ["CLIENTE"]}}
$ curl $GW/api/v1/clientes/me (token de maria)
{"id":3,"nombres":"Maria","apellidos":"Lopez
Huaman","email":"maria.lopez@gmail.com","telefono":"951222333","direccion":"Jr. Puno 210,
Juliaca","activo":true}
```

Caso 2 — Evento duplicado ignorado (primer consumidor)

```
$ reenvio de usuario.registrado usuarioId=5 (misma key y valor)   # 19:37:26
19:37:29.226 INFO  UsuarioEventosConsumer - component=usuario-consumer eventType=usuario.registrado
usuarioId=5 status=RECIBIDO partition=0 offset=1
19:37:29.244 WARN  AltaClienteService - component=alta-cliente-service eventType=usuario.registrado
usuarioId=5 status=IGNORADO motivo="alta ya procesada (evento duplicado)" resultadoPrevio=CREADO
$ psql veterinaria_cliente_db
 clientes_maria          id | usuario_id |         email         | resultado | cliente_id |
fecha_proceso
----------------        ----+------------+-----------------------+-----------+------------+----------
-----------------
              1           1 |          5 | maria.lopez@gmail.com | CREADO    |          3 | 2026-10-
15 19:37:01.013478
```

Caso 3 — Fallo real y compensación aplicada

```
### Preparacion: recepcion (ADMIN) registra a un cliente en mostrador   # 19:37:50
$ curl -X POST $GW/api/v1/clientes -H "Authorization: Bearer <TOKEN ADMIN>" -d
'{"nombres":"Carlos","apellidos":"Ramos Condori","email":"carlos.ramos@gmail.com",...}'
```

```
HTTP/1.1 201  {"id":4,"nombres":"Carlos","apellidos":"Ramos
Condori","email":"carlos.ramos@gmail.com",...}
### Auto-registro con el mismo email   # 19:37:51
HTTP/1.1 202  {"usuarioId":6,"email":"carlos.ramos@gmail.com","estado":"PENDIENTE",...}
### cliente-ms
19:37:52.124 INFO  AltaClienteService - ... usuarioId=6 status=PROCESADO resultado=RECHAZADO
clienteId=null email=carlos.ramos@gmail.com
19:37:52.152 INFO  ClienteEventosProducer - component=cliente-producer eventType=cliente.rechazado
usuarioId=6 status=PUBLICADO topic=cliente-eventos partition=1 offset=0
### auth-ms
19:37:52.156 INFO  ClienteEventosConsumer - component=cliente-consumer eventType=cliente.rechazado
usuarioId=6 status=RECIBIDO partition=1 offset=0
19:37:52.166 INFO  AuthService - component=auth-service eventType=cliente.rechazado usuarioId=6
status=COMPENSADA estado=PENDIENTE->RECHAZADO email=carlos.ramos@gmail.com recurso=email_liberado
motivoRechazo="El email ya pertenece a un cliente registrado en mostrador; requiere verificacion
presencial"
$ curl $GW/api/v1/auth/registro/6   # 19:37:56
{"usuarioId":6,"email":"carlos.ramos@gmail.com","estado":"RECHAZADO","idCliente":null,...}
$ curl -X POST $GW/api/v1/auth/login -d '{"email":"carlos.ramos@gmail.com","password":"carlos123"}'
HTTP/1.1 401  {"error":"Unauthorized","message":"Email o contraseña inválidos",...}
$ psql veterinaria_auth_db                                    $ psql veterinaria_cliente_db
(altas_cliente)
 id |         email          |  estado   | habilitado | id_cliente   usuario_id | resultado | motivo
----+------------------------+-----------+------------+------------  ------------+-----------+-------
-----------------------------------
  5 | maria.lopez@gmail.com  | ACTIVO    | t          |          3            5 | CREADO    |
  6 | carlos.ramos@gmail.com | RECHAZADO | f          |                       6 | RECHAZADO | El
email ya pertenece a un cliente registrado en mostrador; ...
```

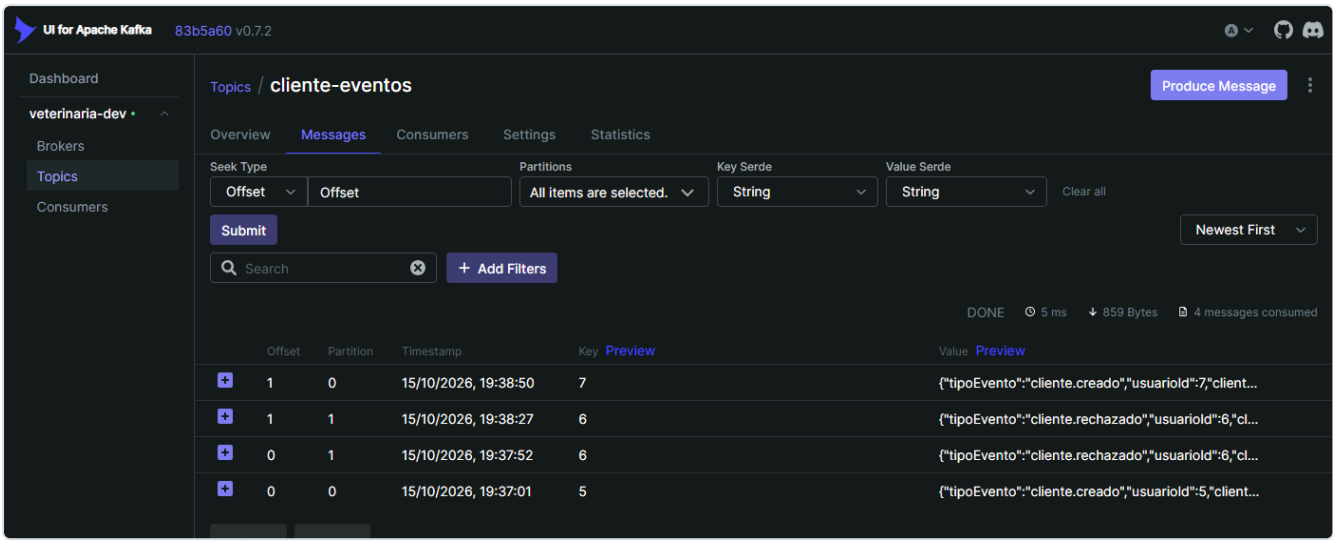
Caso 4 — La compensación probada contra un duplicado

```
$ reenvio de cliente.rechazado usuarioId=6 (misma key y valor)   # 2026-10-15 19:38:25
6|
{"tipoEvento":"cliente.rechazado","usuarioId":6,"clienteId":null,"email":"carlos.ramos@gmail.com","mo
tivo":"El email ya pertenece a un cliente registrado en mostrador; requiere verificacion
presencial","origen":"veterinaria-cliente-ms","timestamp":1792111072125}
19:38:28.039 INFO  ClienteEventosConsumer - component=cliente-consumer eventType=cliente.rechazado
usuarioId=6 status=RECIBIDO partition=1 offset=1
19:38:28.055 WARN  AuthService - component=auth-service eventType=cliente.rechazado usuarioId=6
status=IGNORADO motivo="la cuenta no existe o ya no esta pendiente" estadoActual=RECHAZADO
 id |         email          |  estado   | habilitado
----+------------------------+-----------+------------
  6 | carlos.ramos@gmail.com | RECHAZADO | f

$ reenvio de usuario.registrado usuarioId=6 (alta ya RECHAZADA)   # 19:38:42
19:38:44.657 WARN  AltaClienteService - ... usuarioId=6 status=IGNORADO motivo="alta ya procesada
(evento duplicado)" resultadoPrevio=RECHAZADO
```

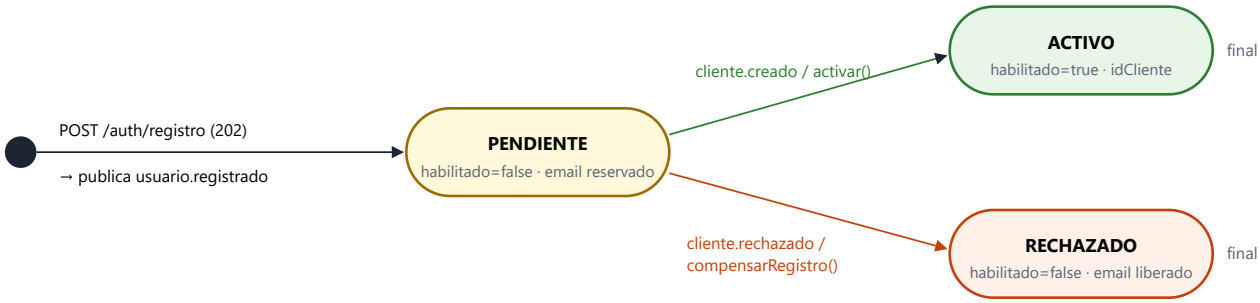
Caso 5 — El recurso liberado se puede volver a usar

```
### Recepcion verifica a Carlos en persona y corrige el email de la ficha de mostrador   # 19:38:49
$ curl -X PUT $GW/api/v1/clientes/4 -H "Authorization: Bearer <TOKEN ADMIN>" -d
'{...,"email":"c.ramos.mostrador@gmail.com",...}'
HTTP/1.1 200
### El mismo email se registra otra vez: 202 (no 409), porque la compensacion lo libero   # 19:38:50
HTTP/1.1 202  {"usuarioId":7,"email":"carlos.ramos@gmail.com","estado":"PENDIENTE",...}
19:38:50.392 INFO  AltaClienteService - ... usuarioId=7 status=PROCESADO resultado=CREADO clienteId=5
email=carlos.ramos@gmail.com
19:38:50.427 INFO  AuthService - ... eventType=cliente.creado usuarioId=7 status=CUENTA_ACTIVADA
idCliente=5 estado=PENDIENTE->ACTIVO
$ login carlos.ramos@gmail.com -> {"email": "carlos.ramos@gmail.com", "idCliente": 5, "realm_access":
{"roles": ["CLIENTE"]}}
$ psql veterinaria_auth_db
 id |         email          |  estado   | habilitado | id_cliente
----+------------------------+-----------+------------+------------
  6 | carlos.ramos@gmail.com | RECHAZADO | f          |
  7 | carlos.ramos@gmail.com | ACTIVO    | t          |          5
         indexname          |  indexdef
----------------------------+-------------------------------------------------------------------------
---------------------
 uk_usuarios_email_vigente | CREATE UNIQUE INDEX ... ON public.usuarios USING btree (email) WHERE
((estado)::text <> 'RECHAZADO'::text)
### Control: un email con cuenta vigente sigue bloqueado   # 19:40:44
$ curl -X POST $GW/api/v1/auth/registro -d '{"email":"maria.lopez@gmail.com",...}'
HTTP/1.1 409  {"error":"Conflict","message":"El email ya está registrado",...}
```



Kafka UI → `cliente-eventos`: `cliente.creado` de los usuarios 5 y 7, `cliente.rechazado` del usuario 6 y su reenvío manual (offset 1, partición 1).

Diagrama de estados de Usuario (cuenta) y tabla de la saga

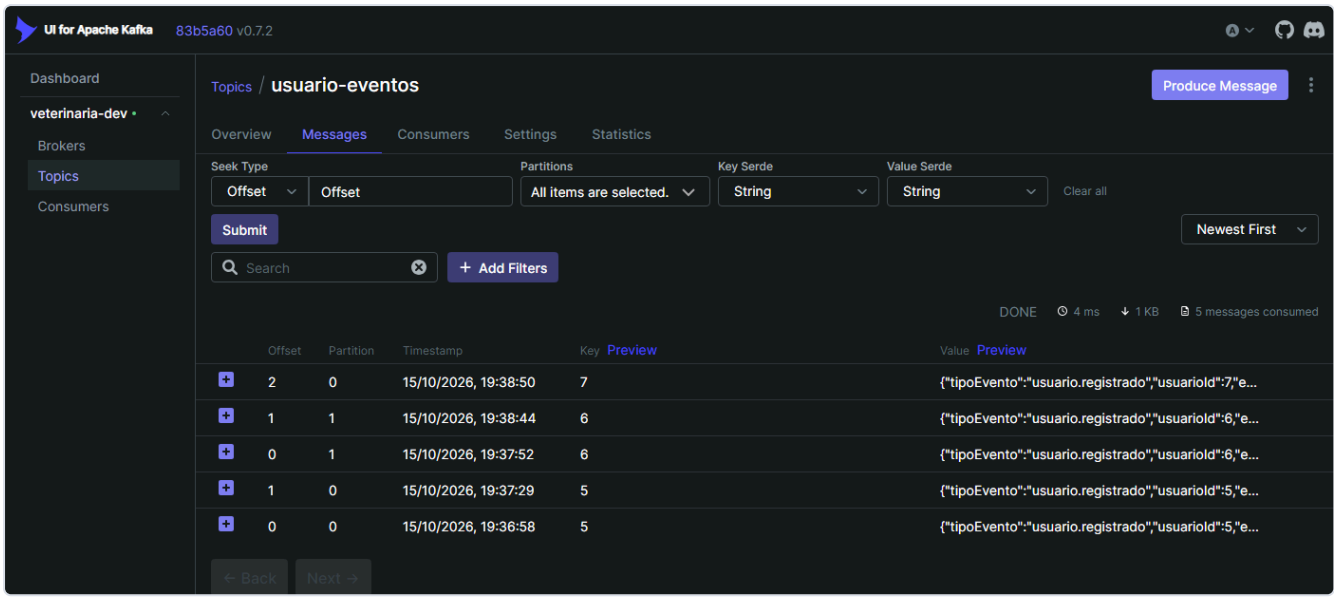


Cuentas sembradas antes de S9 (admin, cliente, cliente2): estado ACTIVO por defecto de la migración V2.

Figura 3. Ciclo de vida de la cuenta. Cualquier evento que llegue con la cuenta fuera de `PENDIENTE` se registra como `IGNORADO`.

Paso de la saga	Servicio	Si falla	Compensación
1. Registrar la cuenta y reservar el email	veterinaria-auth-ms	Email con cuenta vigente: 409, no se publica nada	No aplica
2. Crear el perfil de cliente	veterinaria-cliente-ms	El email ya es de un cliente de mostrador	Registra el alta `RECHAZADO` y publica `cliente.rechazado`
3. Activar la cuenta	veterinaria-auth-ms	— (último paso)	Al recibir `cliente.rechazado`: `PENDIENTE` → `RECHAZADO`, deshabilitada, email liberado

Desde	Evento / acción	Hacia	Efecto
—	`POST /api/v1/auth/registro`	PENDIENTE	Reserva el email, publica `usuario.registrado`
PENDIENTE	`cliente.creado`	ACTIVO	Habilita el login y guarda `idCliente` (claim del JWT)
PENDIENTE	`cliente.rechazado`	RECHAZADO	Compensación: deshabilita y libera el email
ACTIVO / RECHAZADO	cualquier evento repetido	(sin cambio)	`status=IGNORADO`



Kafka UI → `usuario-eventos`: los tres registros (usuarios 5, 6 y 7) y los dos reenvíos manuales de las pruebas de duplicado.

09

Error o hallazgo técnico

HALLAZGO 1 · EL BUG DEJÓ HUELLA EN LOS IDs

En la prueba 3.7 el pago de la cita 9 recibió `id = 26`, cuando el pago anterior era el 5. No faltan datos: cada uno de los 20 INSERT fallidos (10 intentos por cada uno de los dos mensajes duplicados procesados por la versión sin idempotencia) pidió un valor a la secuencia de la columna `IDENTITY` antes de que PostgreSQL rechazara la fila por `uk_pagos_cita_id`, y una secuencia nunca devuelve valores aunque la transacción haga rollback (ids 6 a 25). Conclusión: la restricción UNIQUE protege la integridad, pero no hace gratis al duplicado. Sin la capa 1 cada reentrega cuesta un INSERT fallido, 10 reintentos y un hueco en los ids. La revisión explícita evita todo eso en el caso normal.

HALLAZGO 2 · CAPTURAR DataIntegrityViolationException DENTRO DE @Transactional NO BASTA

La guía captura la excepción dentro de un `procesar()` anotado con `@Transactional`. Al revisarlo se encontró que `save()` participa en la transacción externa: cuando la excepción atraviesa el proxy transaccional del repositorio, la transacción compartida queda marcada *rollback-only*, y además la sesión de Hibernate queda inconsistente después de un error de restricción. El `return` del `catch` no salva nada: al salir, el commit lanza `UnexpectedRollbackException` y Kafka reintenta. Se corrigió de dos formas: en `veterinaria-pago-ms`, `procesar()` ya no es transaccional (`save()` confirma solo y el evento se publica después). En `veterinaria-cliente-ms`, la escritura de dos tablas va en un bean aparte (`AltaClienteTx`) y la excepción se captura afuera.

HALLAZGO 3 · DETUVE EL PROCESO EQUIVOCADO

Para pasar de la versión con el bug a la corregida se usó el PID que devolvió `Start-Process java`, pero ese PID era un lanzador intermedio, no la JVM que escuchaba el puerto 8087. La nueva instancia no pudo arrancar ("Port 8087 was already in use") y el segundo reenvío (offset 6, 19:32:13) lo procesó todavía la versión con el bug, con otros 10 intentos fallidos. Se diagnosticó con `Get-NetTCPConnection -LocalPort 8087` (dueño real: PID 22572), se detuvo esa JVM y se repitió la prueba 3.4 a las 19:33:29. La lección: verificar que el reinicio realmente ocurrió (hora de `Started` y de `partitions assigned`) antes de atribuir un resultado al código nuevo.

> Durante la práctica, los puertos 19092, 18085, 15432 y 15435 estaban ocupados por contenedores de otro proyecto (pagatu). Se detuvieron con `docker stop` durante las pruebas, igual que en S8.

10

Preguntas de defensa

1. ¿Por qué Kafka garantiza "al menos una vez" y no "exactamente una vez"?

Porque el procesamiento del mensaje y la confirmación del offset son dos pasos separados. Si el consumidor procesa el mensaje y se cae, o se reasigna la partición, antes de confirmar el offset, el mensaje se vuelve a entregar. Kafka ofrece transacciones que hacen atómico "leer, escribir en Kafka y confirmar el

offset", pero eso solo cubre lo que vive dentro de Kafka. El INSERT en PostgreSQL queda fuera, salvo que se coordinen ambos sistemas con algo como 2PC. El costo sería más latencia, bloqueos y acoplamiento entre broker y base de datos. Por eso la práctica estándar es aceptar duplicados y hacer idempotente al consumidor.

2. ¿Por qué revisar con findByCitaId y además capturar la excepción del save?

Cada capa cubre un caso distinto. La revisión explícita resuelve el caso común (reentregas en serie) de forma legible y barata: no se intenta un INSERT que fallará (el hallazgo 1 muestra lo que cuesta). Pero entre la revisión y el INSERT hay una ventana: dos entregas simultáneas pueden pasar ambas la revisión. Solo la restricción UNIQUE de la base de datos es atómica frente a esa carrera. Una sola capa deja o un costo alto o un hueco real.

3. Saga orquestada vs. coreografiada, ¿cuál es la de este sistema?

En la orquestada un coordinador central conoce todo el flujo, manda comandos y dispara las compensaciones. Es más fácil de seguir, pero todos los servicios dependen de él. En la coreografiada no hay coordinador: cada servicio reacciona a eventos y publica los suyos. Ambas sagas de este sistema son **coreografiadas**: `citas-ms` no sabe que `pago-ms` existe, solo publica `cita.creada` y reacciona a `pago.validado` o `pago.fallido`. El precio es que el flujo queda repartido entre listeners y se reconstruye siguiendo el `citaId` / `usuarioId` en los logs.

4. ¿Por qué compensar no borra la cita ni la deja en PENDIENTE_PAGO?

Borrarla perdería la trazabilidad: existió una cita y su pago fue rechazado (hay un pago `FALLIDO` que la referencia). Dejarla en `PENDIENTE_PAGO` mentiría: el proceso terminó y ya no llegará ningún pago. `CANCELADA` es un estado final, auditable y coherente con lo que pasó. Además permite que la guardia de idempotencia funcione: un `pago.fallido` repetido ve `CANCELADA` y se ignora. Lo mismo vale para la cuenta `RECHAZADO` de la saga propia.

5. ¿Qué cambiaría con una pasarela real?

`pago-ms` llamaría al API de la pasarela (con un token del comercio y una clave de idempotencia propia, por ejemplo el `citaId`, para no cobrar dos veces) o recibiría su respuesta por webhook firmado. `FALLIDO` o `VALIDADO` saldría de esa respuesta. La decisión nunca puede depender de un valor que envía el cliente: cualquiera podría declarar que su pago está aprobado. `TARJETA_RECHAZADA` solo existe para que la prueba sea determinista.

6. ¿Qué pasaría si mi compensación no tuviera la misma guardia?

Un `cliente.rechazado` repetido o tardío podría llegar después de que la cuenta se activó por otra vía, o después de un nuevo registro con el mismo email, y la deshabilitaría: un cliente activo perdería el acceso. En la saga de pago, un `pago.fallido` repetido devolvería el insumo dos veces e inflaría el inventario. La guardia `estado == PENDIENTE` hace que confirmación y compensación sean mutuamente excluyentes y que cada una ocurra como máximo una vez.

7. ¿Por qué restaurarStock no relanza la excepción si mascota-ms está caído?

Porque la alternativa es peor. Si la compensación se detuviera, la cita quedaría en `PENDIENTE_PAGO` con un pago ya `FALLIDO` del otro lado: un limbo inconsistente que además seguiría ocupando el insumo. Se elige como garantía mínima la compensación de estado (la cita queda `CANCELADA`, lo que realmente pasó) y el stock como efecto secundario que puede quedar pendiente de reconciliar. El log `RESTAURACION_FALLIDA ...`

`accion=requiere_correccion_manual` deja la marca para hacerlo. Lo correcto a futuro sería un reintento o una cola de pendientes.

11

Reflexión técnica

"Exactamente una vez" exige que procesar un mensaje y registrar que se procesó ocurran como un solo hecho atómico. En un sistema distribuido esos dos pasos viven en lugares distintos (el offset en Kafka, el pago en PostgreSQL) y cualquier caída entre ambos obliga a elegir entre perder el mensaje o repetirlo. Coordinarlos con una transacción distribuida bloquea recursos, acopla servicios y no escala. Por eso la práctica es aceptar "al menos una vez" y diseñar el consumidor para que un duplicado no tenga efecto. En la práctica lo vi directamente: sin guardia, un solo mensaje repetido generó 10 intentos y 10 errores. Knight Capital es la misma falla a escala real: un servidor que volvía a ejecutar una acción con cada señal, sin ningún estado que dijera "esto ya lo hice". La idempotencia no es una optimización. Es lo que hace que un sistema que recibe mensajes repetidos siga siendo correcto.

12

Criterios de aceptación, limitaciones y bibliografía

Criterio (4.4)	Evidencia	
Bug de duplicados reproducido (3.2)	Sección 3: 10 intentos contra `uk_pagos_cita_id`	cumple
Idempotencia en dos capas, sin excepción tras el fix (3.3–3.4)	Sección 3: `status=IGNORADO`, `COUNT = 1`	cumple
Fallo de pago simulado y compensación `PENDIENTE_PAGO → CANCELADA`	Sección 4: logs, REST y BD	cumple
Compensación idempotente ante evento repetido	Sección 5 (cita 9) y sección 7 (cita 10, stock sin cambio)	cumple
Proceso propio: revisión + UNIQUE real, evento de compensación propio, guardia, tres casos	Sección 8: casos 1 a 5	cumple
Recurso reservado liberado por la compensación	Sección 8, caso 5 (índice parcial)	cumple
Diagramas de estados con compensación	Figuras 2 y 3, tablas de transiciones	cumple
Parte E (bono): stock antes, durante y después	Sección 7, Tabla 5 reproducida	cumple
Error o hallazgo diagnosticado; reflexión; anexo	Secciones 9 y 11, última página	cumple

Limitaciones conocidas (se dejan abiertas a propósito)

- **Outbox:** si un servicio cae entre el commit y la publicación, el evento se pierde y la cita o la cuenta queda en `PENDIENTE_*`.
- **Sin expiración:** no hay temporizador que lleve a un estado `EXPIRADA` una cita o cuenta de la que nunca llega respuesta.
- **Compensación que falla:** si `restaurar-stock` falla, el stock queda desincronizado hasta una corrección manual (sin reintento ni dead-letter).

- **Reserva sin transacción distribuida:** si `crear()` descuenta el stock y luego falla al guardar la cita, ese stock no se devuelve.
- `GET /api/v1/auth/registro/{id}` es público para poder seguir el estado del registro en las pruebas; en producción debería requerir autenticación.

Bibliografía

U.S. Securities and Exchange Commission. (2013). *In the Matter of Knight Capital Americas LLC* (Release No. 34-70694). https://www.sec.gov/litigation/admin/2013/34-70694.pdf

Richardson, C. (2018). *Microservices Patterns: With Examples in Java* (cap. 4, "Managing transactions with sagas"). Manning.

Richardson, C. (2018). *Pattern: Saga*. microservices.io. https://microservices.io/patterns/data/saga.html

SACAViX. (s. f.). *Saga Pattern*. System Design. https://systemdesign.sacavix.com/patterns/saga

Garcia-Molina, H., y Salem, K. (1987). Sagas. *ACM SIGMOD Record, 16*(3), 249-259. https://doi.org/10.1145/38713.38742

Apache Software Foundation. (2024). *Apache Kafka Documentation*. https://kafka.apache.org/documentation/

Spring for Apache Kafka. (2026). *Reference*. https://docs.spring.io/spring-kafka/reference/

PostgreSQL Global Development Group. (2026). *Partial Indexes*. https://www.postgresql.org/docs/current/indexes-partial.html

Anexo: Feedback de la sesión

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	Que los duplicados no son un error de Kafka sino algo normal en un sistema distribuido, y que la idempotencia se diseña en el consumidor, en dos capas: una revisión del resultado y una restricción UNIQUE que la respalde. También que una compensación es una transición de estado nueva, no un borrado.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	Cómo interactúa la captura de `DataIntegrityViolationException` con `@Transactional`: capturarla dentro del método transaccional no evita que la transacción quede marcada para rollback.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cómo se implementa el Outbox Pattern sin una librería, y cómo se reintenta una compensación que falló (por ejemplo, la devolución de stock)?
Nivel de comprensión de la clase de hoy	☑ ¡Entendido! - Lo domino y podría explicarlo. ☐ Más o menos. ☐ Necesito ayuda.
¿Cómo puedo ayudarte a comprender mejor el tema?	Con un ejemplo en clase de una saga con más de un paso compensable, para comparar coreografía y orquestación.
Autoevaluación de participación y esfuerzo	☑ Muy Comprometido/a: Me esforcé al máximo. ☐ Comprometido/a. ☐ Poco Comprometido/a.
Satisfacción con la clase (1 a 10)	9